

API Security Testing Lab Report

Evary Louis

Florida Atlantic University

March 29th, 2026

1. Introduction

This project demonstrates a hands-on approach to understanding API security using tools such as Postman and Burp Suite, along with a vulnerable web application called OWASP Juice Shop. The purpose of this lab was to understand how web applications communicate through APIs, how requests are structured, and how security analysts inspect and manipulate web traffic.

The lab focused on three main areas:

- Setting up a local testing environment
- Intercepting and analyzing API traffic
- Testing API requests using Postman

All testing was performed in a controlled environment using a local instance of OWASP Juice Shop. No real systems were targeted.

2. Tools Used

- Node.js
Download: <https://nodejs.org/>
- OWASP Juice Shop
Download: <https://github.com/juice-shop/juice-shop/releases>
- Burp Suite
Download: <https://portswigger.net/burp/communitydownload>
- Postman
Download: <https://www.postman.com/downloads/>

3. Environment Setup

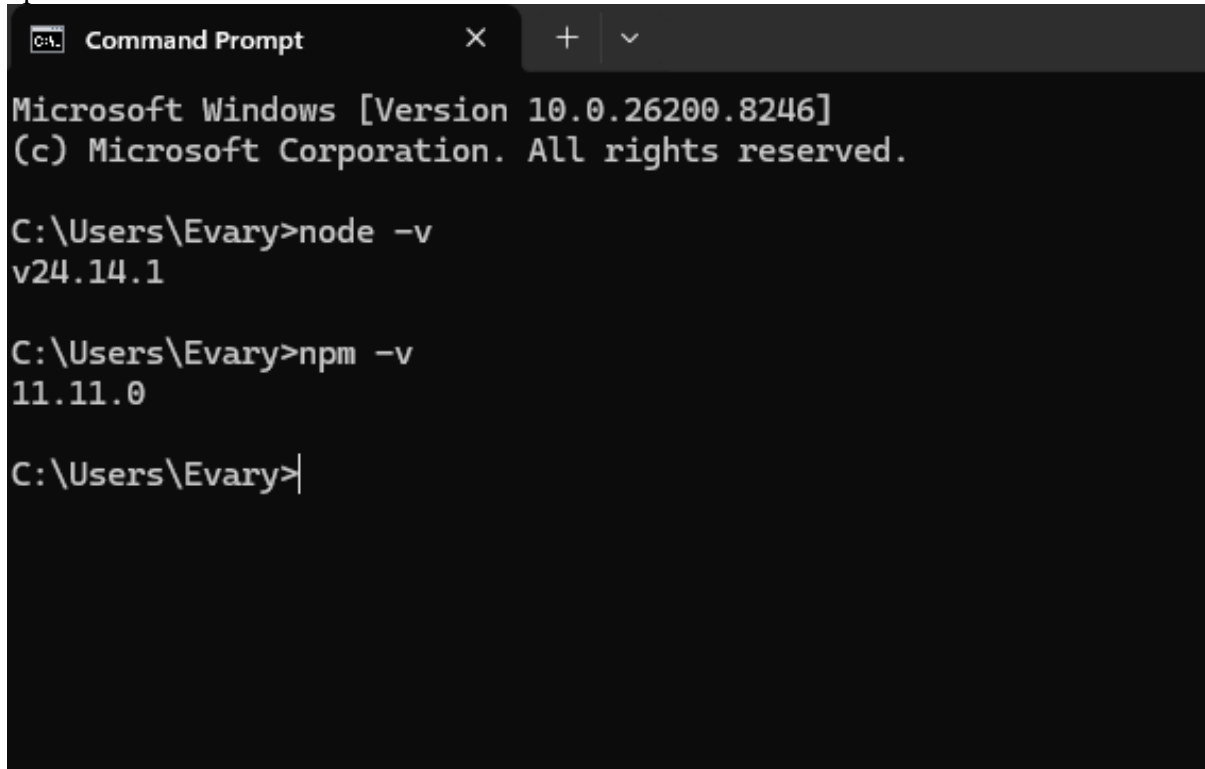
3.1 Installing Node.js

Node.js was installed to run the Juice Shop application locally.

After installation, the following commands were used to verify setup:

```
node -v
```

```
npm -v
```



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Evary>node -v
v24.14.1

C:\Users\Evary>npm -v
11.11.0

C:\Users\Evary>
```

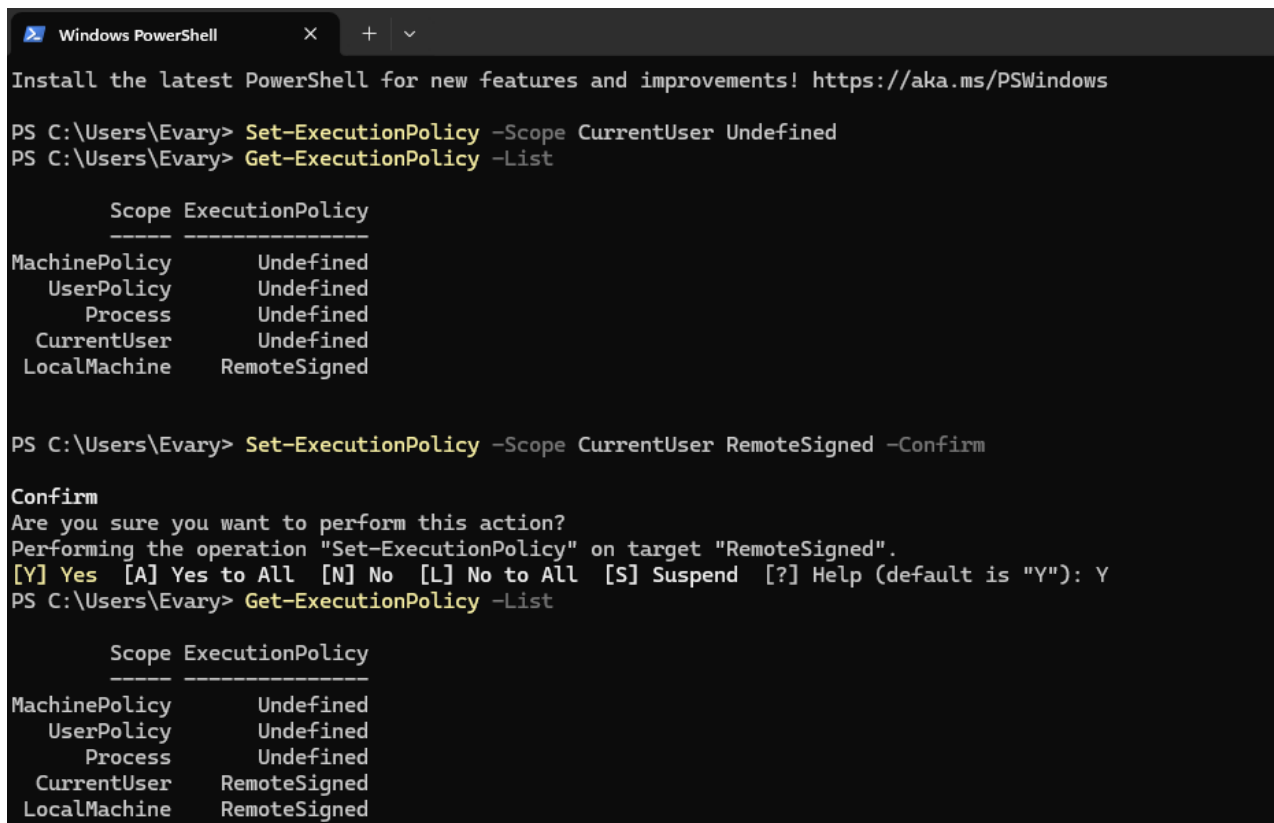
Figure 1: Node.js and npm successfully installed and verified

3.2 Configuring PowerShell Execution Policy

PowerShell execution policy was modified to allow scripts to run locally:

Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

This step ensures that local scripts can execute without being blocked.



```
Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Evary> Set-ExecutionPolicy -Scope CurrentUser Undefined
PS C:\Users\Evary> Get-ExecutionPolicy -List

Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser Undefined
LocalMachine RemoteSigned

PS C:\Users\Evary> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned -Confirm

Confirm
Are you sure you want to perform this action?
Performing the operation "Set-ExecutionPolicy" on target "RemoteSigned".
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): Y
PS C:\Users\Evary> Get-ExecutionPolicy -List

Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser RemoteSigned
LocalMachine RemoteSigned
```

Figure 2: PowerShell execution policy set to RemoteSigned

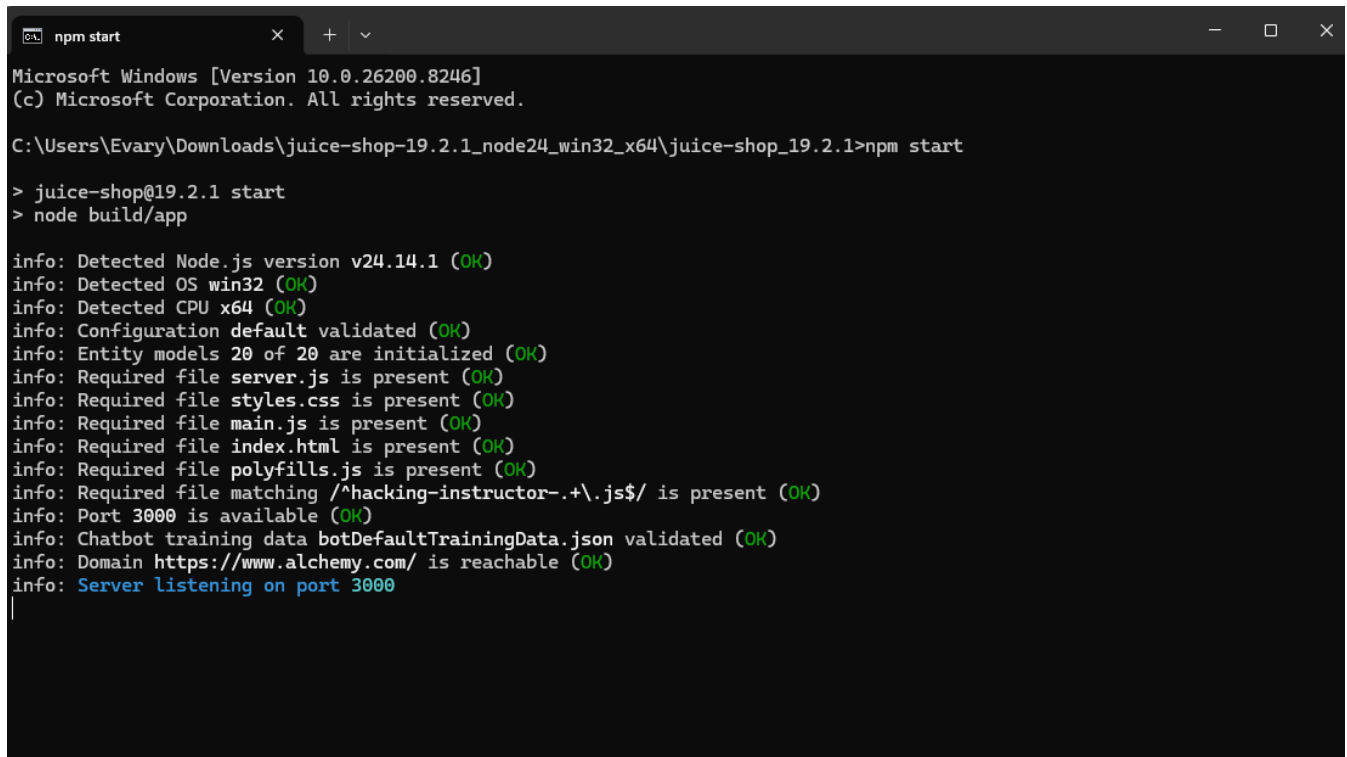
3.3 Running OWASP Juice Shop

The Juice Shop application was downloaded from GitHub and extracted.

The server was started using:

```
npm start
```

The application successfully launched on port 3000.



```
npm start
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Evary\Downloads\juice-shop-19.2.1_node24_win32_x64\juice-shop_19.2.1>npm start

> juice-shop@19.2.1 start
> node build/app

info: Detected Node.js version v24.14.1 (OK)
info: Detected OS win32 (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file index.html is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file matching /^hacking-instructor-.*\.js$/ is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
```

Figure 3: Juice Shop server running locally

3.4 Accessing the Application

The application was accessed through a browser at:

<http://127.0.0.1:3000/>

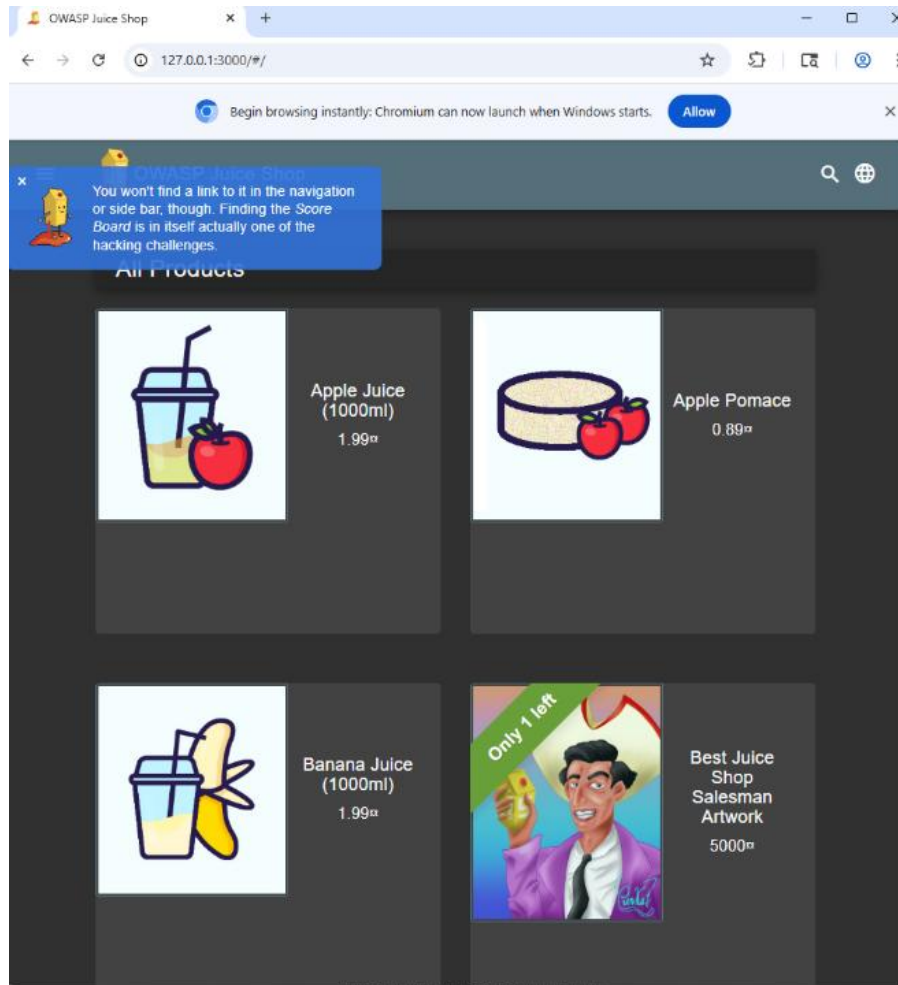


Figure 4: OWASP Juice Shop homepage

4. Login Interaction

A test login attempt was performed to generate API traffic that could be analyzed.

Test credentials were intentionally entered to generate observable API traffic

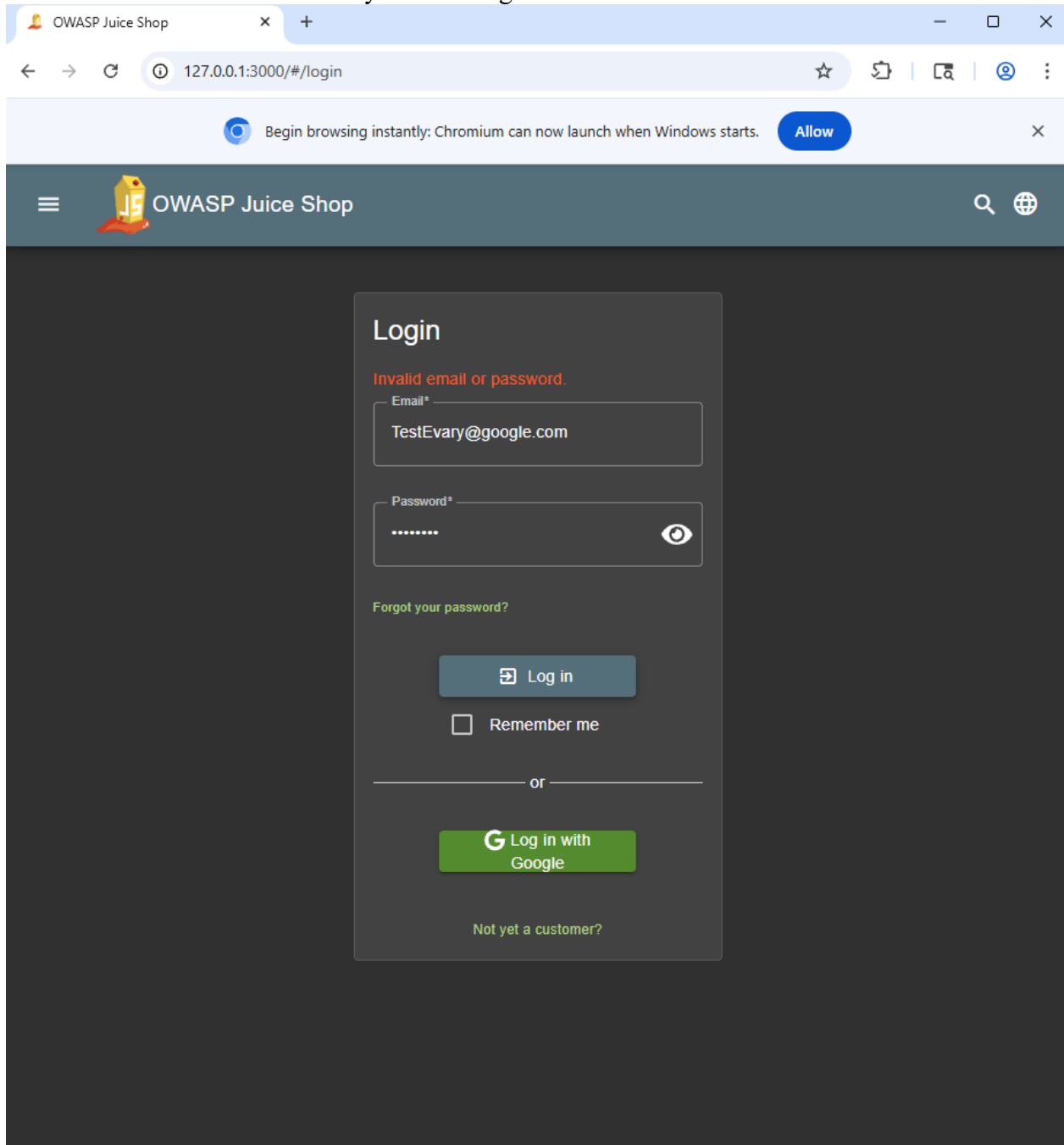


Figure 5: Login attempt using test credentials

5. Burp Suite Traffic Analysis

5.1 Capturing HTTP Traffic

Burp Suite was used as a proxy to capture and inspect all browser requests.

Traffic was monitored under:

Proxy → HTTP History

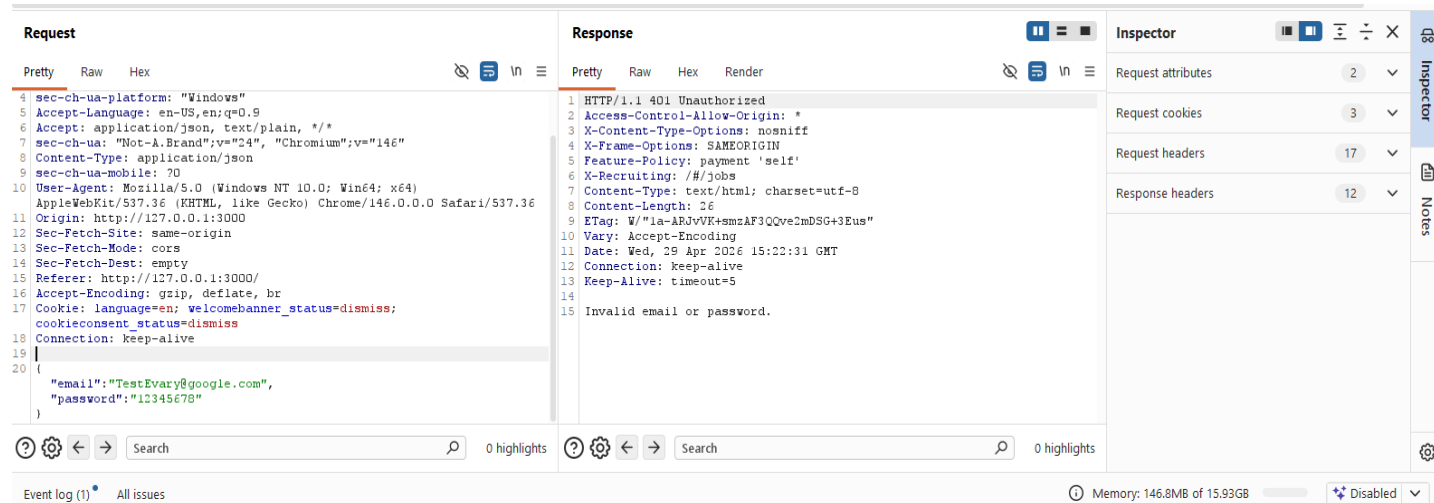


Figure 6: Captured HTTP traffic in Burp Suite

5.2 Filtering Requests

Due to the large number of requests, filters were applied to isolate relevant data.

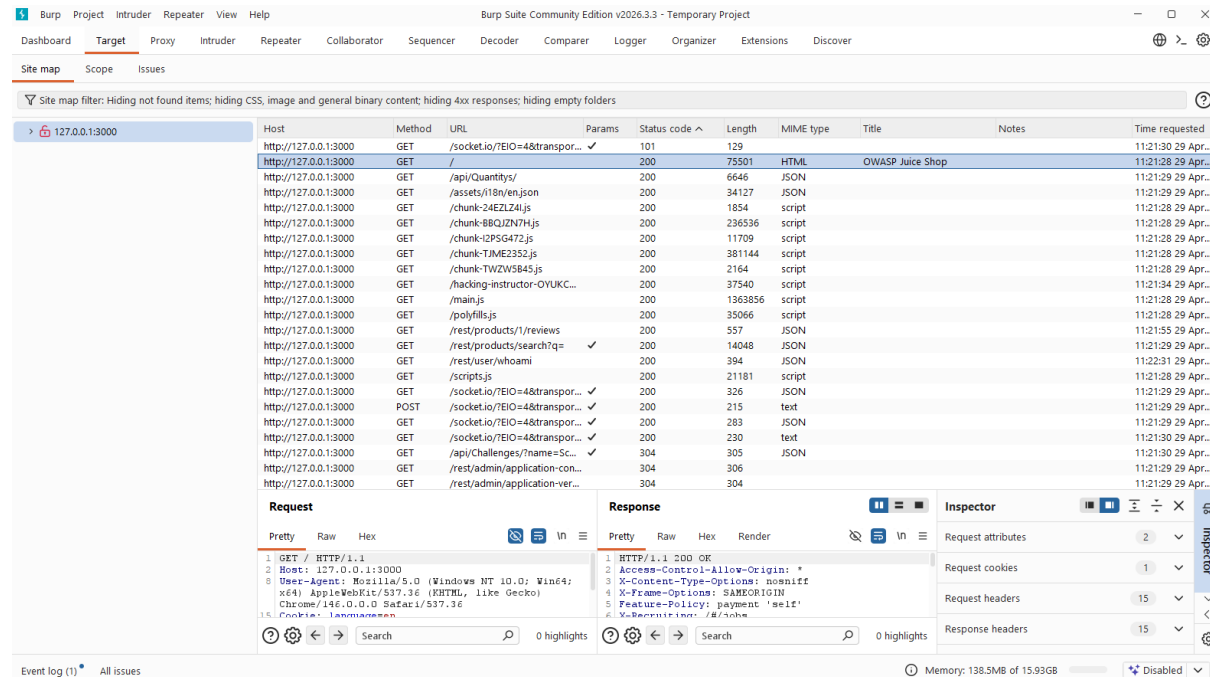


Figure 7: Filtering out unnecessary traffic

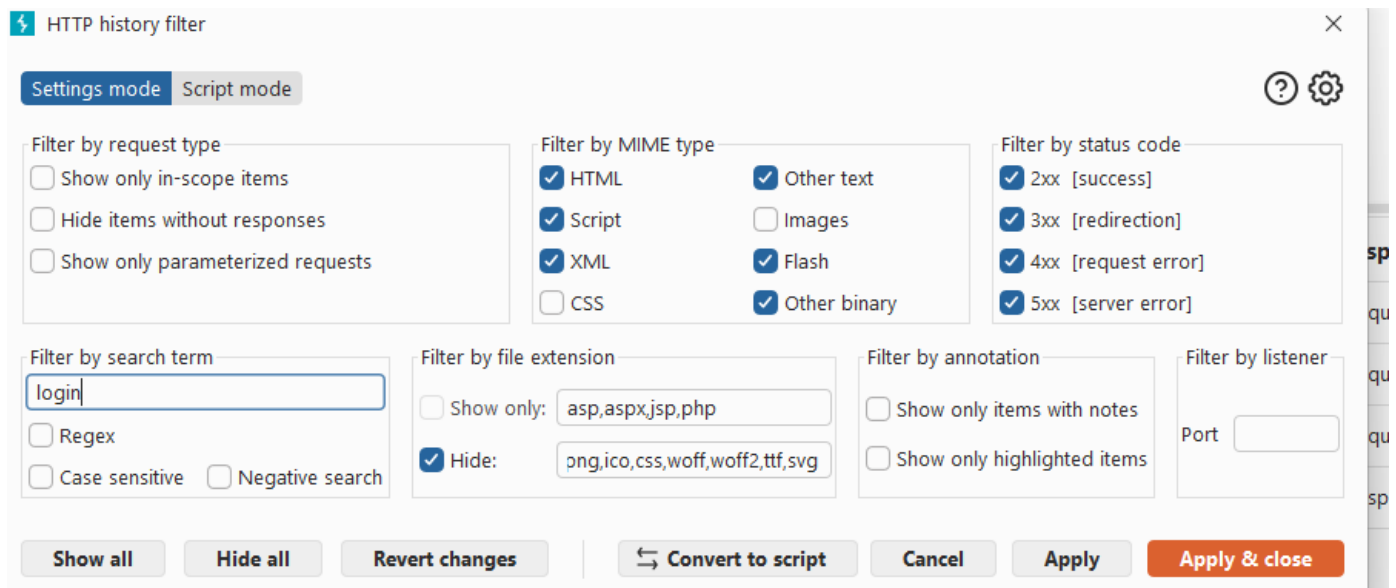


Figure 8: Filtering to locate login request

5.3 Identifying Login API Endpoint

The login request was identified as:

POST /rest/user/login

This request contained user credentials in JSON format.

5.4 Sending Request to Repeater

The request was sent to the Repeater tool for further testing.

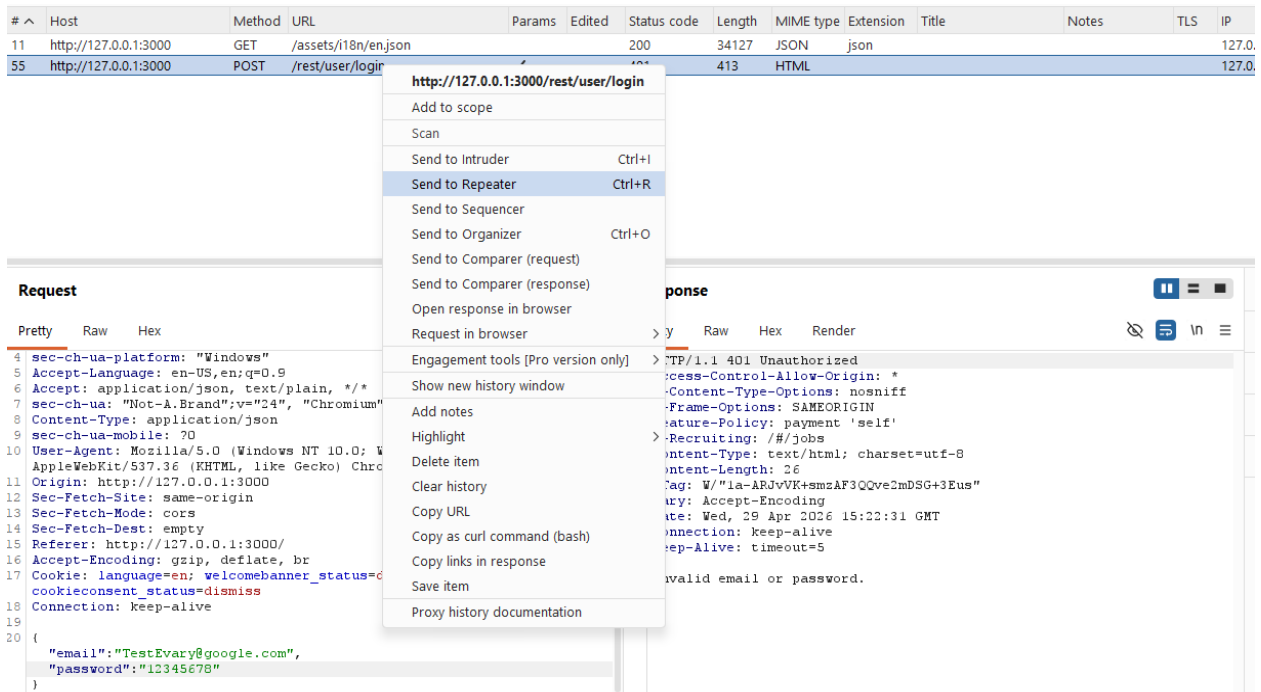
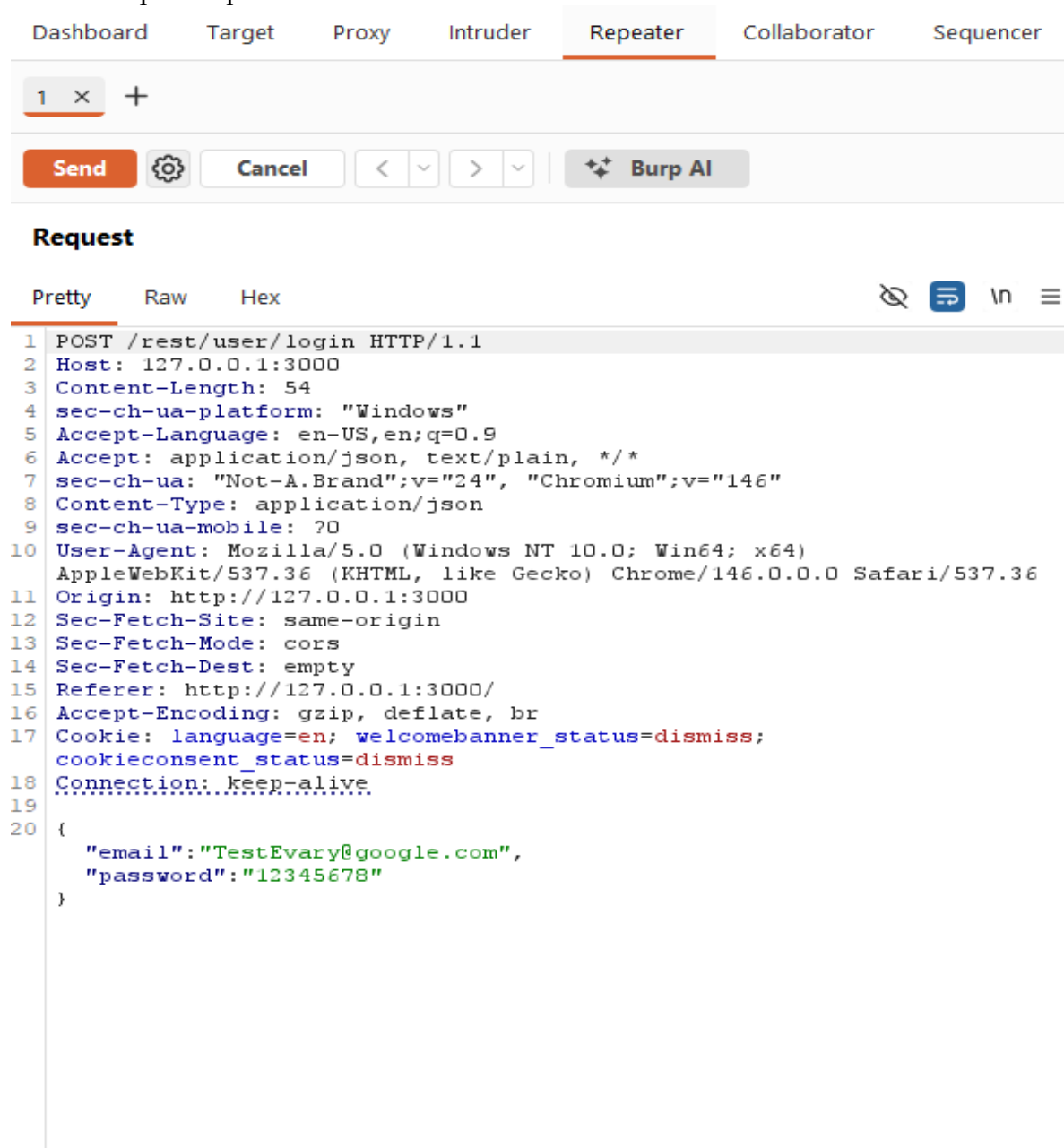


Figure 9: Sending request to Repeater

5.5 Inspecting Request in Repeater

The intercepted request was reviewed in detail.



The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The top navigation bar includes 'Dashboard', 'Target', 'Proxy', 'Intruder', 'Repeater' (highlighted), 'Collaborator', and 'Sequencer'. Below the navigation bar, there is a tab labeled '1' with a close button 'x' and a plus button '+'. A toolbar contains a 'Send' button, a settings gear icon, a 'Cancel' button, navigation arrows, and a 'Burp AI' button. The main area is titled 'Request' and has three tabs: 'Pretty' (selected), 'Raw', and 'Hex'. On the right side of the 'Request' section, there are icons for a search, a list, and a menu. The request details are displayed in a monospaced font, showing a POST request to '/rest/user/login' with various headers and a JSON body.

```
1 POST /rest/user/login HTTP/1.1
2 Host: 127.0.0.1:3000
3 Content-Length: 54
4 sec-ch-ua-platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/plain, */*
7 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/json
9 sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
    AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
11 Origin: http://127.0.0.1:3000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://127.0.0.1:3000/
16 Accept-Encoding: gzip, deflate, br
17 Cookie: language=en; welcomebanner_status=dismiss;
    cookieconsent_status=dismiss
18 Connection: keep-alive
19
20 {
    "email": "TestEvary@google.com",
    "password": "12345678"
}
```

Figure 10: Login request inside Repeater

5.6 Modifying Request

The request was modified to observe how the server responds to altered inputs.

This demonstrates how attackers can manipulate requests before they reach the server.

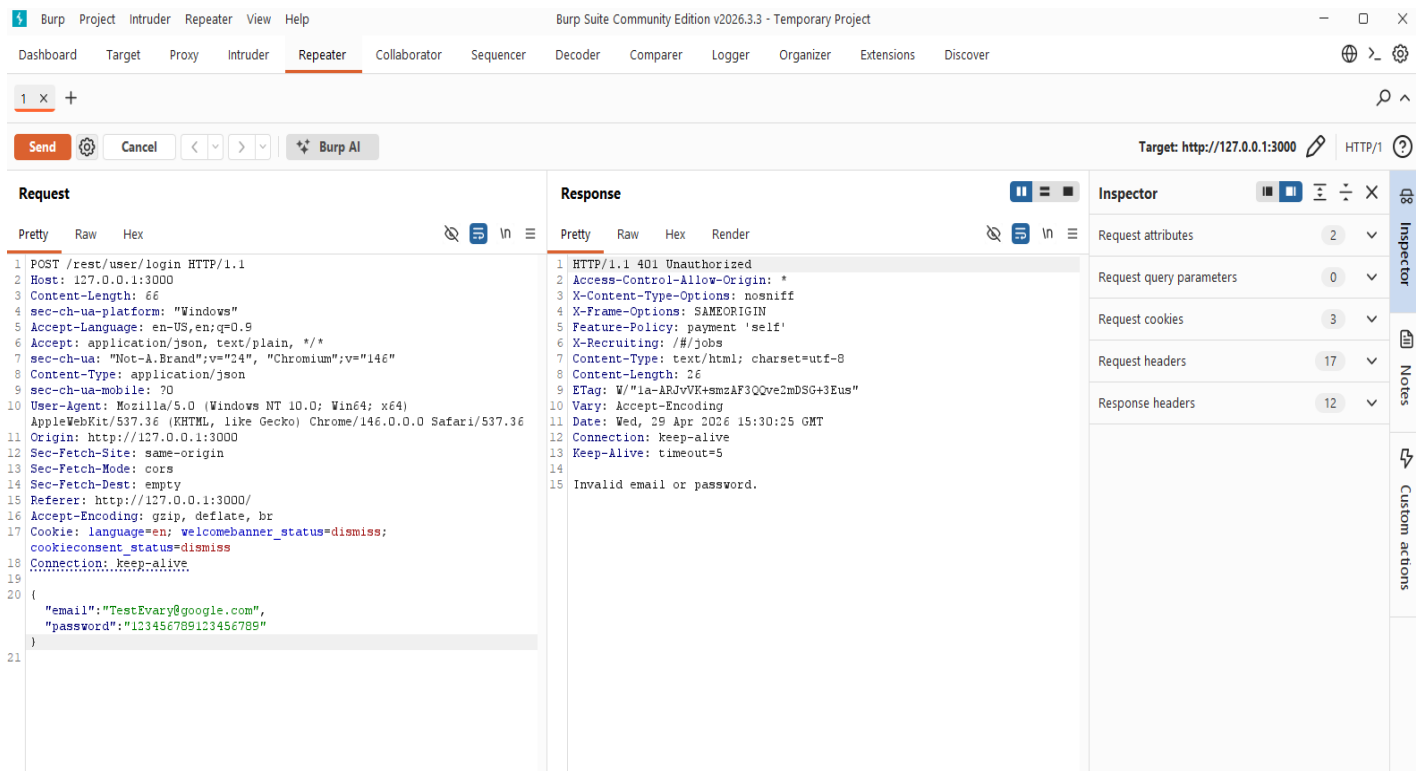


Figure 11: Modified request and server response

6. Security Observations

- User credentials were visible in the request body
- API endpoints can be accessed directly
- Requests can be intercepted and modified
- Server responses reveal validation behavior

These findings highlight the importance of secure API design, including encryption, authentication, and input validation.

APIs should always use HTTPS to prevent interception of sensitive data.

7. API Testing Using Postman

7.1 GET Request

Used to retrieve data from an API:

<https://jsonplaceholder.typicode.com/posts>

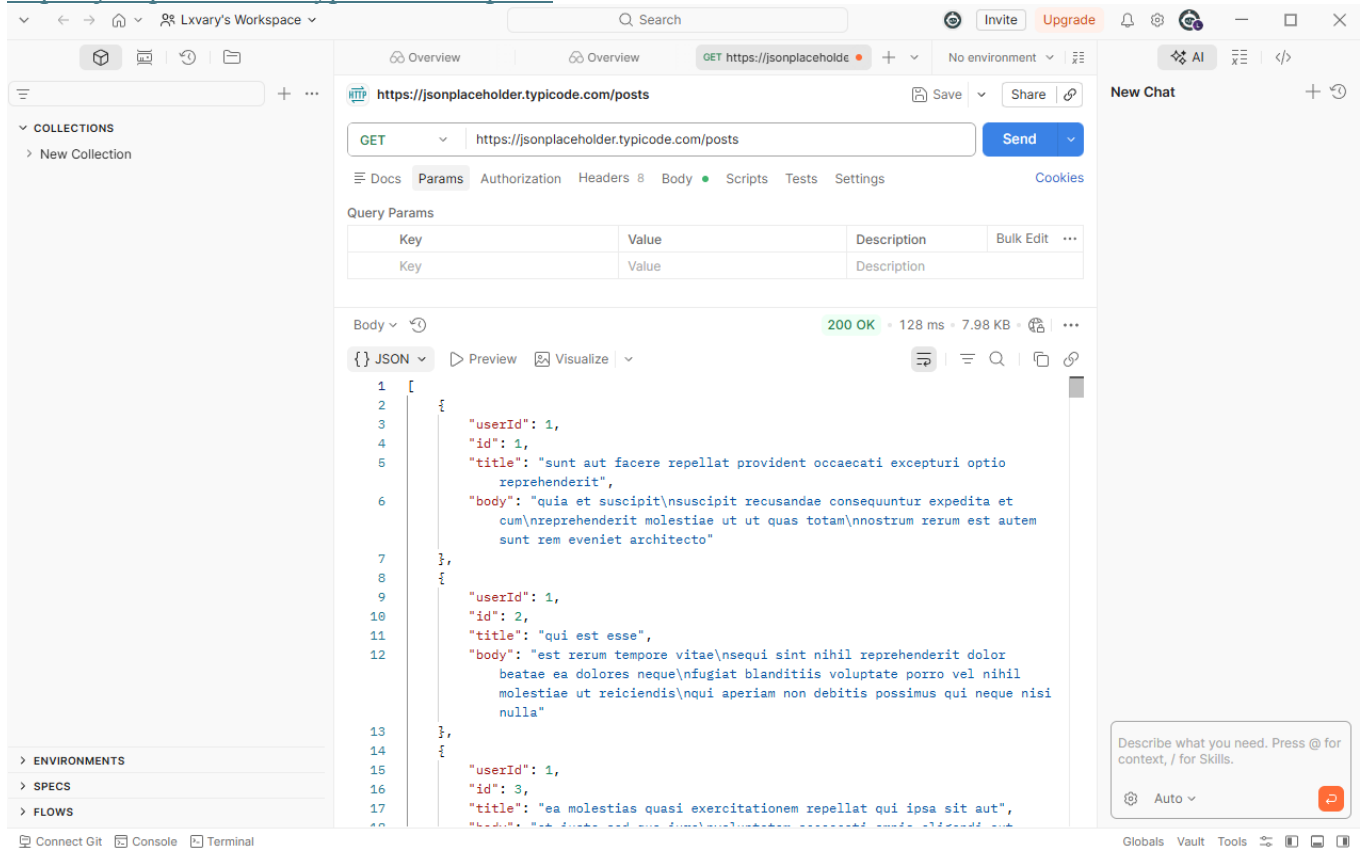


Figure 12: GET request retrieving data

7.2 POST Request

Used to send data:

<https://jsonplaceholder.typicode.com/posts>

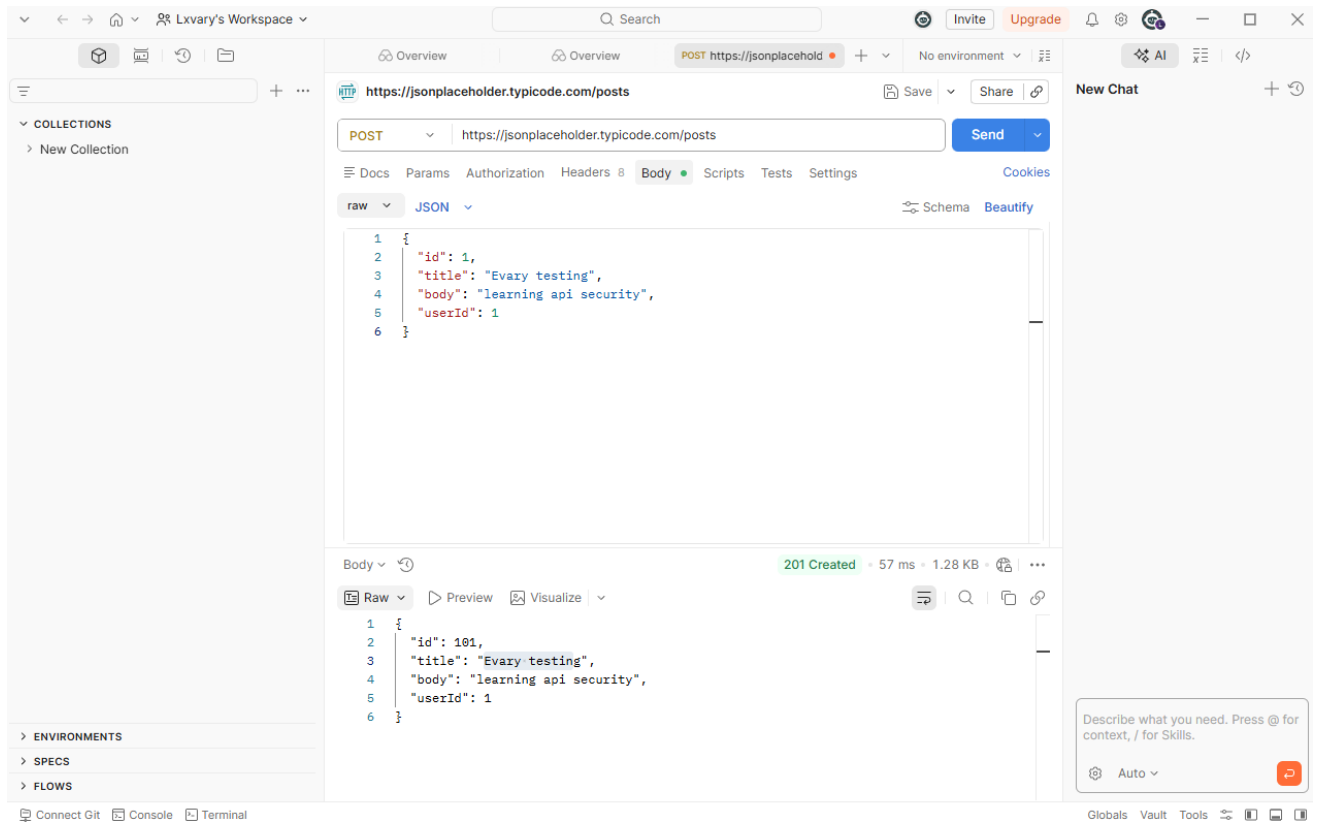


Figure 13: POST request creating data

7.3 PUT Request

Used to update existing data:

<https://jsonplaceholder.typicode.com/posts/1>

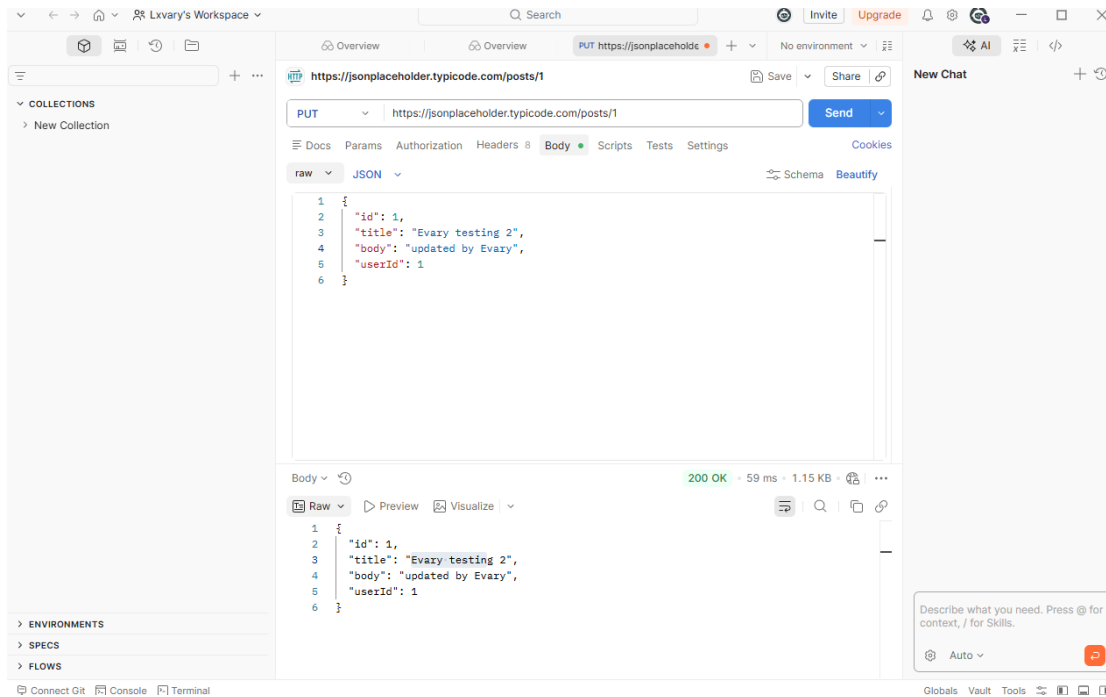


Figure 14: PUT request updating data

8. Conclusion

This lab provided a strong introduction to API security through practical, hands-on experience. By setting up a local testing environment and working with real tools such as Burp Suite and Postman, I was able to understand how web applications communicate through APIs and how sensitive data is transmitted.

One of the most important lessons learned was how easily credentials can be exposed if requests are not properly secured. Intercepting login traffic in Burp Suite showed that user data is sent in structured JSON format, which can be analyzed and modified if proper protections are not in place.

Additionally, working with Postman helped reinforce the understanding of HTTP methods such as GET, POST, and PUT. It also demonstrated how APIs can be tested independently of the user interface.

Throughout the lab, I encountered several setup issues, including incorrect directory usage and execution policy restrictions. Troubleshooting these problems improved my understanding of system configuration and reinforced the importance of attention to detail in cybersecurity work.

Overall, this project strengthened my foundational knowledge of web application security and gave me practical experience with industry tools. It also helped me develop a mindset focused on analyzing, testing, and understanding how systems behave under different conditions.